

자료구조 실습 보고서

[제 12 주] 정렬 성능 비교

제출일 : 2018 년 5 월 25 일

201702081 최재범

1. 프로그램 설명서

a. 주요 알고리즘 / 자료구조

- `public boolean sort(E[] aList, int aSize)` : 속해있는 클래스에 따라서 퀵 정렬 or 삽입 정렬
- 퀵 정렬 (오름차순)
 1. 정렬을 시작하기 이전에, 최댓값을 맨 오른쪽으로 옮겨놓은 후에 이를 제외한 리스트에 대하여 정렬을 시작한다.
 2. 기준이 되는 pivot 원소를 정한다. pivot 원소의 위치는 맨 왼쪽으로 한다.
 3. pivot 에 대하여, 맨 왼쪽과 맨 오른쪽에서부터 좁혀들어오며 탐색을 시작한다.
 4. 왼쪽에서 오른쪽으로 갈 때는 pivot 보다 큰 원소가 있을 때, 오른쪽에서 왼쪽으로 가는 경우는 pivot 보다 작은 원소가 있을 때 탐색을 멈춘다.
 5. 멈춘 인덱스의 원소끼리 교환한다.
 6. 좁혀들어오던 인덱스가 서로 교차되면 탐색을 멈추고, 왼쪽으로 가던 인덱스의 원소와 pivot 을 바꾼다.
 7. pivot 하나에 대하여 정렬이 완료된 리스트에서, pivot 양 쪽의 구간에 대하여 같은 작업을 반복한다.
- 삽입 정렬 (오름차순)
 1. 정렬을 시작하기 이전에, 최솟값을 맨 왼쪽으로 옮겨놓은 후에 이를 제외한 리스트에 대하여 정렬을 시작한다.
 2. 삽입할 원소(`insertedElement`) 를 두 번째 인덱스 부터 탐색하며, 동시에 그 이전 인덱스를 차례차례 내려가면서(`insertionLoc` 인덱스) 해당 위치의 원소가 삽입할 원소보다 클 때마다 둘 사이의 원소를 한 칸씩 뒤로 당긴다.
 3. `insertionLoc` 의 다음 위치에 `insertedElement` 를 넣는다.

b. 함수 설명서

■ AppController

- `public AppController()` : 생성자
- `public void run()` : 프로그램 실행

■ AppView

- `public AppView()` : 생성자
- `public void output(String aString)` : 문자열 출력
- `public void outputLine(String aString)` : 문자열 출력하고 줄바꿈

■ DataGenerator (Static class)

- public static Integer[] ascendingOrderList(int aSize) : 오름차순 리스트 생성
- public static Integer[] descendingOrderList(int aSize) : 내림차순 리스트 생성
- public static Integer[] randomOrderList(int aSize) : 무작위 리스트 생성

- Sort (Abstract class)
 - protected void swap(E[] aList, int i, int j) : 주어진 리스트의 두 인덱스에 위치한 값 교환

- InsertionSort
 - public InsertionSort() : 생성자
 - public boolean sort(E[] aList, int aSize) : 주어진 크기의 리스트를 삽입정렬

- QuickSort
 - public QuickSort() : 생성자
 - public boolean sort(E[] aList, int aSize) : 주어진 크기의 리스트를 퀵 정렬

- ListOrder (Enum)
 - public String orderName() : ListOrder 형 변수에 할당된 리스트 유형에 맞는 한글 이름 반환

- Experiment
 - public Experiment(ParameterSet givenParameterSet) : 생성자 - ParameterSet 설정
 - public long[] durationOfSort(Sort<Integer> aSort, Integer[] experimentList) : 주어진 정렬 방법을 이용하여, 주어진 리스트를 정렬

- ExperimentManager : 정렬 측정 테스트 명령
 - public ExperimentManager() : 생성자 - ParameterSet 을 기본값으로 설정
 - public ParameterSet parameterSet() : Getter - ParameterSet 반환
 - public long measuredResultForInsertionSortAt(int sizeStep) : Getter - 주어진 순번의 삽입정렬 측정 결과 반환
 - public long measuredResultForQuickSortAt(int sizeStep) : Getter - 주어진 순번의 퀵정렬 측정 결과 반환
 - public void prepareExperiment(ParameterSet aParameterSet) : 측정에 필요한 ParameterSet 과 대상 리스트 설정
 - public void performExperiment(ListOrder anOrder) : 주어진 리스트 종류에 대하여, 삽입 정렬과 퀵 정렬한 뒤 저장

- ParameterSet : 측정 테스트의 반복 시행에 관한 정보
 - public ParameterSet(int givenStartingSize, givenNumberOfSizeIncreasingSteps, int givenIncrementSize) : 생성자 - 시작 측정 크기, 실험할 총 횟수, 단계별 측정 크기 증가량 설정
 - public int startingSize() : Getter - 시작 측정 크기 반환
 - public int numberOfSizeIncreasingSteps() : Getter - 실험할 총 횟수 반환
 - public int incrementSize() : Getter - 단계별 측정 크기 증가량 반환

- public void setStartingSize(int aStartingSize) : Setter - 시작 측정 크기 설정
- public void setNumberOfSizeIncreasingSteps(int aNumberOfSizeIncreasingSteps) : Setter - 실험할 총 횟수 설정
- public void setIncrementSize(int aSetIncrementSize) : Setter - 단계별 측정 크기 증가량 설정
- public int maxDataSize() : 최대 측정 크기 계산하여 반환

■ Timer (Final class)

- public Timer() : 생성자
- public void start() : 측정 시작
- public void stop() : 측정 종료
- public long duration() : 측정에 걸린 시간 반환 (밀리초)

c. 종합 설명서

저번 정렬 검증 프로그램을 정렬 시간 측정 프로그램으로 수정하였다.

오름차순 리스트, 내림차순 리스트, 무작위 리스트를 각각 생성하여

삽입 정렬과 퀵 정렬의 두 가지 방법으로 정렬하는 데 걸린 시간을 측정한다.

2. 프로그램 장단점 분석

장점 : 정렬과 관련된 QuickSort, InsertionSort 클래스에서 중복되는 기능을 Sort 추상 클래스로부터 상속받았기 때문에, 쓸데없는 코드의 중복을 막았다.

단점 : x

3. 실행 결과 분석

a. 입력과 출력

```
<<< 정렬 성능 비교 프로그램을 시작합니다. >>>

>> 2 가지 정렬의 성능 비교 : 삽입, 퀵 << > 내림차순 데이터를 사용하여 실행한 측정 :
> 오름차순 데이터를 사용하여 실행한 측정 :
    <Insertion Sort>    <Quick Sort>    [ 1000]    12    13
[ 1000]    1    67 [ 2000]    62    3
[ 2000]    1    64 [ 3000]    81    8
[ 3000]    1    9  [ 4000]    37    14
[ 4000]    1    20 [ 5000]    54    22
[ 5000]    1    46 [ 6000]    78    33
[ 6000]    2    46 [ 7000]   105    48
[ 7000]    2    51 [ 8000]   140    61
[ 8000]    2    60 [ 9000]   161    80
[ 9000]    1    69 [10000]   223   109
[10000]    2    89
```

```

> 무작위 데이터를 사용하여 실행한 측정 :
      <Insertion Sort>    <Quick Sort>
[ 1000]                1             2
[ 2000]                5             1
[ 3000]                8             1
[ 4000]               16             1
[ 5000]               23             1
[ 6000]               41             2
[ 7000]               57             3
[ 8000]               64             5
[ 9000]               95             4
[10000]               98             5

<<< 정렬 성능 비교 프로그램을 종료합니다. >>>

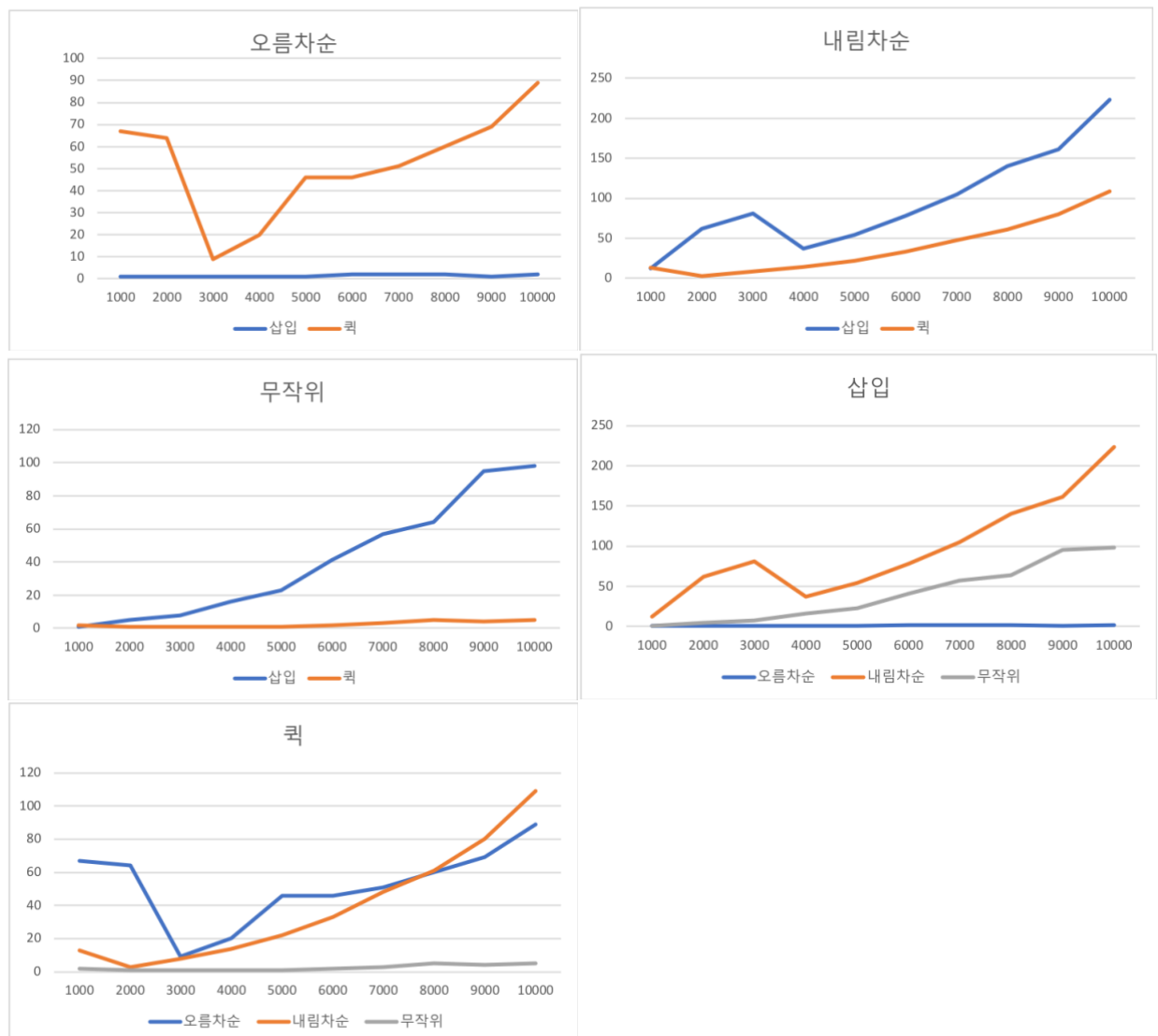
```

b. 결과분석

지난 번 과제에서 정렬이 올바르게 되었음을 검증하였고, 이번 과제에서 정렬 데이터에 관한 부분은 전혀 수정하지 않았기 때문에, 올바르게 정렬이 되었다고 할 수 있다.

시간을 그래프로 그려보면 다음과 같다.

측정 초 단위를 나노초로 하였더니 세로축의 간격 간 차이가 매우 커서, 그래프의 변화가 매우 급격하게 나타났기 때문에 밀리초로 하였다.



오름차순 : 삽입 정렬이 퀵 정렬보다 우세

내림차순 : 퀵 정렬이 삽입 정렬보다 우세

무작위 : 퀵 정렬이 삽입 정렬보다 우세

삽입 : 오름차순, 무작위, 내림차순 순으로 우세

퀵 : 무작위가 제일 우세, 오름차순과 내림차순 비슷

삽입 정렬은 오름차순일 때 제일 우세하며, 이 때의 시간복잡도는 $O(n)$ 이다. 반면 내림차순일 때 제일 불리하며, 이 때의 시간복잡도는 $O(n^2)$ 이다.

삽입 정렬 그래프의 경우에서 거시적으로 봤을 때,

내림차순에서는 그래프가 아래로 볼록하게 위로 솟구치는 경향이 있었고 이는 $O(n^2)$ 에 부합한다고 볼 수 있다.

오름차순에서는 매우 빨라 잘 드러나지 않았지만 거의 직선의 형태를 띠므로 $O(n)$ 에 부합한다고 볼 수 있다.

무작위 리스트는 내림차순과 오름차순의 중간 정도에 위치하였다.

퀵 정렬은 오름차순이나 내림차순으로 이미 정렬이 되어있을 때 제일 불리하며, 이 때의 시간복잡도는 $O(n^2)$ 이다. 평균적으로는 $O(n \log n)$ 의 복잡도를 보인다.

퀵 정렬 그래프의 경우에서 거시적으로 봤을 때,

내림차순과 오름차순에서는 그래프가 아래로 볼록하게 위로 솟구치는 경향을 보였으며 이는 $O(n^2)$ 에 부합한다고 볼 수 있다.

무작위 리스트에서는 매우 서서히 증가하는 형태로 드러났으며 이는 $O(n \log n)$ 에 부합한다고 볼 수 있다.

4. 소스코드

AppController.java

```
public class AppController {

    // Private instance variables
    private AppView _appView;
    private ExperimentManager _manager;

    // Constructor
    public AppController() {
        this._appView = new AppView();
        this._manager = new ExperimentManager();
    }
}
```

```

// ***** RUN *****
public void run() {
    this._appView.outputLine("<<< 정렬 성능 비교 프로그램을 시작합니다. >>>");
    this._appView.outputLine("");

    {
        this._appView.outputLine("> 2 가지 정렬의 성능 비교 : 삽입, 퀵 <<");
        this._manager.prepareExperiment(null); // Use Default ParameterSet
        this.measureAndShowResultFor(ListOrder.Ascending);
        this.measureAndShowResultFor(ListOrder.Descending);
        this.measureAndShowResultFor(ListOrder.Random);
    }

    this._appView.outputLine("<<< 정렬 성능 비교 프로그램을 종료합니다. >>>");
}

// Private Methods

private void showTableTitle(ListOrder anOrder) {
    this._appView.outputLine(
        "> " + anOrder.orderName() + " 데이터를 사용하여 실행한 측정 : "
    );
}

private void showTableHead() {
    this._appView.outputLine(
        String.format("%8s", " ") +
        String.format("%16s", "<Insertion Sort>") +
        String.format("%16s", "<Quick Sort>")
    );
}

// Show Results when measurement is finished
private void showMeasuredResult() {

    int startingSize = this._manager.parameterSet().startingSize();
    int incrementSize = this._manager.parameterSet().incrementSize();
    int numberOfSteps = this._manager.parameterSet().numberOfSizeIncreasingSteps();

    for(int step = 0; step < numberOfSteps; step++) {
        int sortingSize = startingSize + (incrementSize * step);
        this._appView.outputLine(
            "[" + String.format("%5d", sortingSize) + "]" +
            String.format("%16d", this._manager.measuredResultForInsertionSortAt(step)) +
            String.format("%16d", this._manager.measuredResultForQuickSortAt(step))
        );
    }
}

// Start measurement when Data is prepared
private void measureAndShowResultFor(ListOrder anOrder) {
    this.showTableTitle(anOrder);
    this.showTableHead();
    this._manager.performExperiment(anOrder);
}

```

```

        this.showMeasuredResult();
        this._appView.outputLine("");
    }

}

```

AppView.java

// 출력 담당

```

public class AppView {

    public AppView() { }

    public void output(String aString) {
        System.out.print(aString);
    }

    public void outputLine(String aString) {
        System.out.println(aString);
    }

}

```

DataGenerator.java

// 데이터 생성

// static class를 구현하기 위하여, 모든 멤버는 static으로 선언하고 클래스는 final로

import java.util.Random;

```

public final class DataGenerator {

    // 인스턴스 생성 불가
    private DataGenerator() {}

    // 오름차순 리스트 생성
    public static Integer[] ascendingOrderList(int aSize) {
        Integer[] list = null;

        if(aSize > 0) {
            list = new Integer[aSize];
            for(int i = 0; i < aSize; i++) {
                list[i] = i;
            }
        }

        return list;
    }

    // 내림차순 리스트 생성
    public static Integer[] descendingOrderList(int aSize) {
        Integer[] list = null;

        if(aSize > 0) {
            list = new Integer[aSize];
            for(int i = 0; i < aSize; i++) {
                list[i] = aSize - 1 - i;
            }
        }
    }
}

```



```

    }

    }

    return list;
}

// 무작위 리스트 생성
public static Integer[] randomOrderList(int aSize) {
    Integer[] list = null;

    if(aSize > 0) {
        list = new Integer[aSize];
        for(int i = 0; i < aSize; i++) {
            list[i] = i;
        }

        Random random = new Random();
        for(int i = 0; i < aSize; i++) {
            int j = random.nextInt(aSize);
            Integer temp = list[i];
            list[i] = list[j];
            list[j] = temp;
        }
    }

    return list;
}
}

```

ExperimentManager.java

// Manages Experiment with Required Data

```

public class ExperimentManager {

    // Constants for the Parameters :
    private static final int DEFAULT_NUMBER_OF_SIZE_INCREASING_STEPS = 10;
    private static final int DEFAULT_INCREMENT_SIZE = 1000;
    private static final int DEFAULT_STARTING_SIZE = DEFAULT_INCREMENT_SIZE;

    // Constants for Sorting Methods :
    private static final InsertionSort<Integer> INSERTION_SORT = new InsertionSort<Integer>();
    private static final QuickSort<Integer> QUICK_SORT = new QuickSort<Integer>();

    // Private instance variables :
    private Experiment _experiment;
    private ParameterSet _parameterSet;

    private Integer[] _ascendingList; // Ascending Data List to Sort
    private Integer[] _descendingList; // Descending Data List to Sort
    private Integer[] _randomList; // Random Data List to Sort

    private long[] _measuredResultForInsertionSort; // To Store Measurement Results for Insertion
    Sort
    private long[] _measuredResultForQuickSort; // To Store Measurement Results
    for Quick Sort

    // Constructor
    public ExperimentManager() {

```

```

        this.setParameterSetWithDefaults(); // Set Parameter Set to Default Value
    }

    // Public Getters
    public ParameterSet parameterSet() {
        return this._parameterSet;
    }

    public long measuredResultForInsertionSortAt(int sizeStep) { // Return Measured Results
        return this._measuredResultForInsertionSort[sizeStep];
    }

    public long measuredResultForQuickSortAt(int sizeStep) {
        return this._measuredResultForQuickSort[sizeStep];
    }

    // Prepare All Data (Fields) For Experiment
    public void prepareExperiment(ParameterSet aParameterSet) {

        if(aParameterSet != null) {
            this._parameterSet = aParameterSet; // Set New ParameterSet
        }
        this._experiment = new Experiment(this._parameterSet);

        this.prepareExperimentLists(); // Set Lists to Experiment
    }

    // TEST START : List Order should be specified
    public void performExperiment(ListOrder anOrder) {

        // 1. Set List order
        Integer[] experimentList = this.experimentListOfOrder(anOrder);

        // 2. Measure
        this._measuredResultForInsertionSort = this._experiment.durationsOfSort(INSERTION_SORT, experimentList);
        this._measuredResultForQuickSort = this._experiment.durationsOfSort(QUICK_SORT, experimentList);
    }

    // Private Methods

    // Generate Lists for Experiment and Set
    private void prepareExperimentLists() {
        int maxDataSize = this._parameterSet.maxDataSize();

        this._ascendingList = DataGenerator.ascendingOrderList(maxDataSize);
        this._descendingList = DataGenerator.descendingOrderList(maxDataSize);
        this._randomList = DataGenerator.randomOrderList(maxDataSize);
    }

    // Set ParameterSet to default : called by constructor
    private void setParameterSetWithDefaults() {
        this._parameterSet = new ParameterSet(DEFAULT_STARTING_SIZE,
        DEFAULT_NUMBER_OF_SIZE_INCREASING_STEPS,

```

```

DEFAULT_INCREMENT_SIZE);
    }

    // Return List matches with Parameter anOrder
    private Integer[] experimentListOfOrder(ListOrder anOrder) {
        switch(anOrder) {
            case Ascending:
                return this._ascendingList;
            case Descending:
                return this._descendingList;
            default:
                return this._randomList;
        }
    }
}

```

Experiment.java

// Start experiment when given ParameterSet, Data to Sort, Sorting Method

```

public class Experiment {

    // Private instance variables
    private final ParameterSet _parameterSet;

    // Constructor
    public Experiment(ParameterSet givenParameterSet) {
        this._parameterSet = givenParameterSet;
    }

    // Repeat sorting given list as much as steps (copy list to test several times)
    public long[] durationsOfSort(Sort<Integer> aSort, Integer[] experimentList) {

        int numberOfSteps = this._parameterSet.numberOfSizeIncreasingSteps();
        long[] durations = new long[numberOfSteps];

        int sortingSize = this._parameterSet.startingSize();
        int incrementSize = this._parameterSet.incrementSize();

        for(int step = 0; step < numberOfSteps; step++) {

            // Copy as much as size to sort : from experimentList
            Integer[] listForSorting = this.copyListOfGivenSize(experimentList, sortingSize);

            // Sort, Measure, Store
            durations[step] = this.durationOfSingleSort(aSort, listForSorting);

            sortingSize += incrementSize;
        }

        return durations;
    }

    // Private Methods

```

```

// Copy Integer List as much as you want to copy
private Integer[] copyListOfGivenSize(Integer[] aList, int copiedSize) {
    Integer[] copiedList = null;
    if(copiedSize <= aList.length) {
        copiedList = new Integer[copiedSize];
        for(int i = 0; i < copiedSize; i++) {
            copiedList[i] = aList[i];
        }
    }
    return copiedList;
}

// Sort given List and return duration
private long durationOfSingleSort(Sort<Integer> aSort, Integer[] aList) {
    Timer timer = new Timer();
    timer.start();
    {
        aSort.sort(aList, aList.length);
    }
    timer.stop();
    return timer.duration();
}
}

```

Sort.java

```

// 정렬의 기본 베이스
// 추상 클래스 : 인스턴스 생성할 때 반드시 상속이 필요

public abstract class Sort<E extends Comparable<E>> {

    protected void swap(E[] aList, int i, int j) {
        E tempElement = aList[i];
        aList[i] = aList[j];
        aList[j] = tempElement;
    }

    // 생성자
    protected Sort() {}

    // 정렬
    public abstract boolean sort(E[] aList, int aSize);
}

```

InsertionSort.java

```

// 삽입 정렬

public class InsertionSort<E extends Comparable<E>> extends Sort<E> {

    // 생성자
    public InsertionSort() {}

    // 삽입 정렬
    public boolean sort(E[] aList, int aSize) {

        if( (aSize < 1) || (aSize > aList.length) ) {
            return false;
        }
    }
}

```

```

// 최솟값 위치 탐색 후 0 번 인덱스로 이동
int minLoc = 0;
for(int i = 1; i < aSize; i++) {
    if( aList[i].compareTo(aList[minLoc]) < 0 ) {
        minLoc = i;
    }
}
this.swap(aList, 0, minLoc);

// 리스트 탐색해가며 맞는 위치에 삽입
for(int i = 2; i < aSize; i++) {
    E insertedElement = aList[i]; // 대상 원소를 미리 저장해놓음 (원래 위치에서 덮어쓰워짐)
    int insertionLoc = i - 1;

    // 삽입할 원소보다 더 크다면 거기서부터 원소 한칸씩 뒤로 당김 (앞쪽의 공간 생성)
    while(aList[insertionLoc].compareTo(insertedElement) > 0) {
        aList[insertionLoc + 1] = aList[insertionLoc];
        insertionLoc--;
    }

    // 삽입할 원소보다 더 작은 원소를 가리키게 됨
    // 최솟값은 0번 인덱스에 존재하기 때문에 끝까지 가도 벗어나
    aList[insertionLoc + 1] = insertedElement;
}

return true;
}
}

```

QuickSort.java

```

public class QuickSort<E extends Comparable<E>> extends Sort<E> {

    // 생성자
    public QuickSort() {}

    public boolean sort(E[] aList, int aSize) {
        if(aSize < 1 || aSize > aList.length) {
            return false;
        }
        else {

            // 최댓값 위치 탐색 후 맨 끝 인덱스로 이동
            int maxLoc = 0;
            for(int i = 0; i < aSize - 1; i++) {
                if( aList[i].compareTo(aList[maxLoc]) > 0 ) {
                    maxLoc = i;
                }
            }

            if( aList[maxLoc].compareTo(aList[aSize - 1]) > 0 ) {
                this.swap(aList, aSize - 1, maxLoc);
            }
            this.quickSortRecursively(aList, 0, aSize - 2);
            return true;
        }
    }
}

```

```

// 비공개 함수

// pivot 위치 결정 : 왼쪽으로
private int pivot(E[] aList, int left, int right) {
    return left;
}

// partition 수행 후 pivot 위치 반환
private int partition(E[] aList, int left, int right) {

    int pivot = this.pivot(aList, left, right);
    int toRightIndex = left + 1;
    int toLeftIndex = right;

    while(toRightIndex <= toLeftIndex) {

        // toRightIndex : pivot 보다 작을 시 맞는 위치 (오름차순)
        while( aList[toRightIndex].compareTo(aList[pivot]) < 0 ) {
            toRightIndex++;
        }

        // toLeftIndex : pivot 보다 클 시 맞는 위치 (오름차순)
        while( aList[toLeftIndex].compareTo(aList[pivot]) > 0 ) {
            toLeftIndex--;
        }

        if(toRightIndex < toLeftIndex) {
            this.swap(aList, toRightIndex, toLeftIndex);
        }
    }

    this.swap(aList, toLeftIndex, pivot);

    // pivot 위치 반환
    return toLeftIndex;
}

// 재귀정렬
private void quickSortRecursively(E[] aList, int left, int right) {
    if(left < right) {
        int mid = this.partition(aList, left, right);
        this.quickSortRecursively(aList, left, mid - 1);
        this.quickSortRecursively(aList, mid + 1, right);
    }
    // 정렬이 끝난 상태라면 아무것도 하지 않음
}
}

```

ListOrder.java

// 리스트의 종류 구분

public enum ListOrder {

Ascending, // 오름차순 리스트 유형
 Descending, // 내림차순 리스트 유형

```

Random;                // 무작위 리스트 유형

public static final String[] ORDER_NAMES = { "오름차순", "내림차순", "무작위" };

// ListOrder 인스턴스가 가진 리스트 유형의 한글명 반환
public String orderName() {
    return ListOrder.ORDER_NAMES[this.ordinal()];
}

}

```

ParameterSet.java

```

public class ParameterSet {

    // Private instance variables
    private int _startingSize;
    private int _numberOfSizeIncreasingSteps;
    private int _incrementSize;

    // Constructor
    public ParameterSet( int givenStartingSize,
                        int givenNumberOfSizeIncreasingSteps,
                        int givenIncrementSize )
    {
        this._startingSize = givenStartingSize;
        this._numberOfSizeIncreasingSteps = givenNumberOfSizeIncreasingSteps;
        this._incrementSize = givenIncrementSize;
    }

    // Getters & Setters
    public int startingSize() { return this._startingSize; }
    public int numberOfSizeIncreasingSteps() { return this._numberOfSizeIncreasingSteps; }
    public int incrementSize() { return this._incrementSize; }

    public void setStartingSize(int aStartingSize) {
        this._startingSize = aStartingSize;
    }
    public void setNumberOfSizeIncreasingSteps(int aNumberOfSizeIncreasingSteps) {
        this._numberOfSizeIncreasingSteps = aNumberOfSizeIncreasingSteps;
    }
    public void setIncrementSize(int aSetIncrementSize) {
        this._incrementSize = aSetIncrementSize;
    }

    // return Maximum Data Size
    public int maxDataSize() {
        return this.startingSize() + ( this.incrementSize() * ( this.numberOfSizeIncreasingSteps() - 1 ) );
    }

}

```

Timer.java

```

public final class Timer {

```

```
// Private instance variables
private long _startTime;
private long _stopTime;

// Constructor
public Timer() {};

// Public Methods
public void start() {
    this._startTime = System.currentTimeMillis();
}
public void stop() {
    this._stopTime = System.currentTimeMillis();
}
public long duration() {
    return ( this._stopTime - this._startTime );
}

}
```

DS1_12_201702081_최재범.java

```
public class DS1_12_201702081_최재범 {

    public static void main(String[] args) {

        AppController appController = new AppController();
        appController.run();

    }

}
```